Lab 15 — Storage and StatefulSets

Theory — how a cluster that throws its Pods away still keeps a database: volumes that survive, a storage class that manufactures them, and a controller that gives Pods a name and a disk they get back.

Objectives

- Understand the **PersistentVolume → PersistentVolumeClaim → Pod** chain, and the role of the **StorageClass**.
- Read access modes, the reclaim policy, and what `local-path` does on k3s.
- Tell a **Deployment** from a **StatefulSet**: stable identity, one volume per Pod, *headless* Service.
- Deploy PostgreSQL as a StatefulSet and check that the data survives the Pod's deletion.
- Know how to back up, and know when a database has no business being in the cluster.

---

1. The problem: disposable Pods, data that is not

In lab 06, the rule was: data that must survive is not *inside* the container. In the cluster it is worse: a Pod is disposable (lab 11), replaced at every rollout (lab 12), and may land on **another node** where the previous one's local disk does not exist. An `emptyDir` (a volume tied to the Pod's lifetime) or a `hostPath` (a node directory, lab 11) do not meet that need. Storage is required that exists **independently** of the Pod and of the node, and that the cluster can attach to the Pod wherever it is.

> **→ DOCKER RECAP**
> The named volume of lab 06 (`podman volume create pgdata`) becomes a **PersistentVolume**; the `-v pgdata:/var/lib/postgresql/data` becomes a **PersistentVolumeClaim** referenced by the template; and `podman volume export` becomes... the same thing, a `pg_dump` or a copy of the volume. What changes: the volume can be a network disk (EBS, Azure Disk, NFS, Ceph) attached to the node by a driver, and the cluster manages the life cycle.

2. Three objects for one disk

Object	Who writes it	What it says
StorageClass	the platform team (or the distribution)	"here is how to manufacture a volume": the driver (*provisioner*), its parameters, the reclaim policy, when to create
PersistentVolumeClaim (PVC)	**you**, in the application's namespace	"I want 500 Mi, read-write, of the default class"
PersistentVolume (PV)	the *provisioner*, automatically	"here is a 500 Mi volume, on this disk, bound to that PVC"

The Pod only knows the **PVC** (`persistentVolumeClaim: {claimName: demo-data}`). The StorageClass turns the request into a real volume: that is **dynamic provisioning**, which replaced the era when an administrator created PVs by hand. `kubectl get pvc` shows `Pending` until a volume is bound, then `Bound` with the PV's name (`pvc-<uuid>`).

Two StorageClass settings matter day to day. `volumeBindingMode: WaitForFirstConsumer`: the volume is created only when a Pod uses the PVC — which allows creating it on the right node; a lone PVC therefore stays `Pending`, and that is not an error. `reclaimPolicy: Delete`: deleting the PVC deletes the PV **and its data**; `Retain` keeps the PV orphaned, to be cleaned up by hand. **Access modes** say who may mount: `ReadWriteOnce` (one node at a time — a disk), `ReadOnlyMany`, `ReadWriteMany` (several nodes — NFS, a network filesystem).

> **K3S**
> k3s's default StorageClass, `local-path`, creates a plain directory on the node (`/var/lib/rancher/k3s/storage/pvc-<uuid>_<namespace>_<pvc>`) and marks the PV with a **node affinity**: the Pod can never be scheduled elsewhere. On a single node, this is invisible; on a real cluster, it is exactly what you do not want for a database, and you use a **CSI** driver (*Container Storage Interface*) towards network storage: `gp3` on EKS, `managed-csi` on AKS, Longhorn or Ceph on bare machines. The PVC manifest does not change; the StorageClass does.

> **→ LINUX**
> Under the PVC there is always a **mount**: the kubelet attaches the disk (or directory) to the node, formats it if needed (`ext4` by default), and mounts it into the container like a bind mount from lab 06 — with the same UID questions: PostgreSQL writes as UID 70, and `fsGroup` in the Pod's `securityContext` serves to hand the volume to the right group when the driver does not.

3. Deployment or StatefulSet

A Deployment with a PVC works for **one** instance of an application that tolerates losing a Pod and getting another, provided the two never run at the same time on the same disk. For a database, two things are missing:

- **A stable identity.** A Deployment's Pods are called `api-6c7bb54fc5-2th65`; a **StatefulSet**'s are called `db-0`, `db-1`, `db-2`, in order, and a recreated Pod **takes its name back**.
- **One volume per Pod.** With `volumeClaimTemplates`, each Pod receives **its own** PVC, named `data-db-0`, `data-db-1`..., reattached to it at every recreation. A three-replica Deployment would share the same PVC (impossible with `ReadWriteOnce`, catastrophic for a database).

Add an **ordered start and stop** (`db-0` before `db-1`), an update **one Pod at a time** in reverse order, and the **headless Service** (`clusterIP: None`): instead of a virtual IP that distributes, DNS returns the Pods' IPs directly, and each Pod has its own name, `db-0.db.lab15.svc.cluster.local`. That is what lets a replica `db-1` know that its primary is `db-0`.

> **REMEMBER**
> Deleting a StatefulSet **does not delete** its PVCs: the data stays, and a new StatefulSet of the same name finds it again. Scaling down to `replicas: 0` does not either. It is the symmetrical safety net of `podman compose down` from lab 09 — and it is why a deleted namespace takes the PVCs with it (and, with `Delete`, the data): read `kubectl get pvc` first.

> **→ POSTGRESQL**
> The official image wants a `PGDATA` that is **empty or already initialised**, and `lost+found` (created by `ext4` at the root of a disk) makes it fail: hence the `PGDATA=/var/lib/postgresql/data/pgdata` subdirectory in the StatefulSet. And a StatefulSet at `replicas: 3` does **not** make three PostgreSQL replicas: it makes three independent databases. Replication is the job of an **operator** (CloudNativePG, Zalando, Crunchy) that manages primary, replicas, failover and backups — that is what is deployed in production, not a bare StatefulSet.

4. Backing up, again

The lab 06 rule does not change: **logical** with `kubectl exec db-0 -- pg_dump -U postgres > dump.sql` (consistent while running), **physical** through a volume *snapshot* (`VolumeSnapshot`, if the CSI driver allows it — `local-path` does not) or a copy with the Pod stopped. An operator automates both to object storage. And the question "where is the data?" has a two-command answer: `kubectl get pvc` (which PVC), `kubectl get pv <name> -o yaml` (which disk, which node, which policy).

5. In the workplace

- **The database is often not in the cluster**: a managed service (RDS, Cloud SQL, Azure Database) provides backups, replication and upgrades without anyone touching them; the application reaches it through an `ExternalName` Service or a ConfigMap holding the URL. That is the default choice of teams that do not want to operate PostgreSQL.
- **Inside the cluster, an operator**, never a hand-written StatefulSet, and a network StorageClass with snapshots.
- **"Home-made" StatefulSets** remain legitimate for test environments, caches, queues, and for understanding what the operator does on your behalf.
- **The PVC is the boundary**: the application asks for space, the platform decides where it comes from.

Remember

- Pod → PVC → PV ← StorageClass: the application asks, the class manufactures, the cluster binds.
- `WaitForFirstConsumer` = a lone PVC stays `Pending`; `Delete` = deleting the PVC deletes the data.
- `local-path` with k3s = a node directory with affinity; in production, a network CSI driver.
- StatefulSet = stable names (`db-0`), one PVC per Pod (`data-db-0`), ordered start, headless Service (`db-0.db`).
- Deleting the StatefulSet or scaling it to zero keeps the PVCs; deleting the namespace takes them.
- Three replicas of a PostgreSQL StatefulSet = three databases; replication is an operator's job.
- Backup: `pg_dump` through `kubectl exec`, CSI snapshots, or an operator — and often, a managed database outside the cluster.

Vocabulary

PersistentVolume (PV): a real volume, cluster object. — **PersistentVolumeClaim (PVC)**: a storage request by the application. — **StorageClass**: recipe for manufacturing a volume (*provisioner*, parameters, policy). — **dynamic provisioning**: PV creation on demand. — **`WaitForFirstConsumer`**: volume creation at the first Pod. — **`reclaimPolicy`**: `Delete` or `Retain` when the PVC is deleted. — **access modes**: `ReadWriteOnce`, `ReadOnlyMany`, `ReadWriteMany`. — **CSI**: interface of storage drivers. — **StatefulSet**: controller with stable identity and one volume per Pod. — **`volumeClaimTemplates`**: PVC template per Pod. — **headless Service**: `clusterIP:`

`None`, one DNS name per Pod. — **operator**: specialised controller that operates a stateful application.